

План выполнения работы

1. Изучить лекционный материал, текущее задание, и подготовить CSV-файл для загрузки.

Для работы был выбран файл из датасета Synthetic Order Records, содержащий 10 000 000 записей.

В датасете заказов есть следующие поля:

- Уникальный последовательный идентификатор заказа `order_id`, который является целым числом от 1 до максимального количества записей в файле.
- Идентификатор пользователя `person_id`, который ссылается на покупателя и также является целым числом, но его значение ограничено максимальным значением `order_id`, то есть в файле на 10 000 строк `person_id` будет случайным числом от 1 до 10 000.
- Дата создания заказа `order_date` представлена в строковом формате YYYY-MM-DD.
- Текущий статус заказа `status` может принимать одно из семи значений, например Pending, Shipped или Delivered.
- Общая сумма заказа `total_amount` представляет собой число с плавающей точкой в диапазоне от 10 до 1500 долларов.
- Код валюты `currency` может быть USD, EUR или GBP.
- Способ оплаты `payment_method` выбирается из семи вариантов, включая `credit_card`, `paypal` и `klarna`.
- Метод доставки `shipping_method` может быть одним из восьми вариантов, например `standard`, `express` или `fedex`.
- В поле `notes` содержатся дополнительные заметки по заказу, например просьба оставить доставку у двери, однако это поле заполнено примерно только в 5 процентах записей.

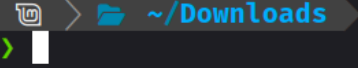
2. Скопировать CSV-файл на ETL-сервер кластера.

В папке /data на ETL-сервере кластера создал папку csv, в которой будем хранить CSV-файлы, затем туда был скопирован CSV-файл, который был описан в 1 пункте этой лабораторной:

```

> docker exec -it etl mkdir -p /data/csv
> cd Downloads
> ls
'2026.05.05_Пятунин Станислав.md'          Отчет_ПР2_2.pdf
bike_sharing.csv                          'Отчет_ПР2_3 (1).pdf'
greenplum-db-6.24.6-rhel7-x86_64.tar.gz    'Отчет_ПР2_3 (2).pdf'
order_10000000.csv                        Отчет_ПР2_3.pdf
Отчет_ПР_1_1.pdf                         'Станислав_CV (1).docx'
Отчет_ПР_1_2.pdf                         'Станислав_CV (1).pdf'
Отчет_ПР_1_3.pdf                         'Станислав_CV (2).pdf'
Отчет_ПР_1_4.pdf                         Станислав_CV.docx
Отчет_ПР_2_1.pdf                         Станислав_CV.pdf
> docker cp order_10000000.csv etl:/data/csv/
Successfully copied 796MB (transferred 796MB) to etl:/data/csv/
> docker exec -it etl ls -la /data/csv/
total 777052
drwxr-xr-x 2 root    root          4096 May 31 11:54 .
drwxrwxr-x 3 gpadmin gpadmin      4096 May 31 11:52 ..
-rw-rw-r-- 1      1000 gpadmin 795687201 May 31 09:59 order_10000000.csv

```



3. Запустить экземпляр gpfdist на ETL-сервере кластера, указав в параметрах путь к каталогу с CSV-файлом.

Средство gpfdist является ближайшим аналогом команды COPY, и предназначено для загрузки данных в распределенные по сегментам таблицы.

В функции ETL-сервера входит хранение импортируемых/экспортируемых данных и «раздача» этих данных по запросам с сегментов. Для этого на ETL-сервере запускается один или несколько экземпляров утилиты gpfdist, которая является web-службой, к которой обращаются сегменты с запросами на данные.

Я использовал следующую команду для запуска:

```
docker exec -d etl /usr/local/greenplum-db-6.24.6/bin/gpfdist -p 8081 -d /data/csv -l /data/gpfdist.log
```

Основные параметры – номер порта, путь к каталогу с файлами с данными, и путь к файлу лога.

А затем проверили, что процесс запустился через команду:

`docker exec etl ps aux | grep gpfdist`

```
> docker exec -d etl /usr/local/greenplum-db-6.24.6/bin/gpfdist -p 8081 -d /data/csv -l /data/gpfdist.log
> docker exec etl ps aux | grep gpfdist
root      228   0.0  0.0 56424  4424 ?        Ss   12:10   0:00 /usr/local/greenplum-db-6.24.6/bin/gpfdist -p 8081 -d /data/csv -l /data/gpfdist.log

~/gp_cluster at 03:10:14
```

Утилита `gpfdist` никаким интерфейсом для управления ею не обладает; для прекращения ее работы необходимо прекратить соответствующий процесс.

4. Создать `external`-таблицу для доступа к данным на основе `gpfdist`-протокола. Имена полей взять из CSV-файла, типы полей рекомендуется указать как `text` для минимизации возможных ошибок.

Для выполнения этого пункта написан следующий скрипт:

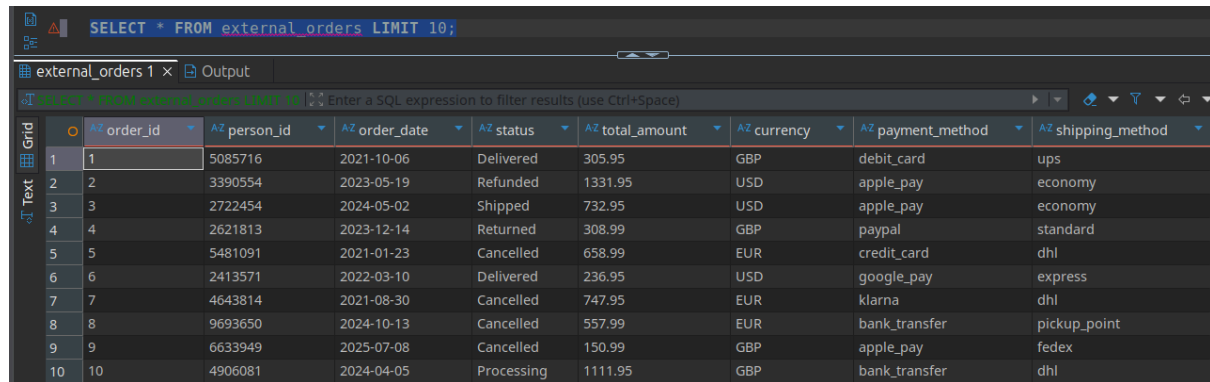
```
CREATE READABLE EXTERNAL TABLE external_orders (
    order_id TEXT,
    person_id TEXT,
    order_date TEXT,
    status TEXT,
    total_amount TEXT,
    currency TEXT,
    payment_method TEXT,
    shipping_method TEXT,
    notes TEXT
)
LOCATION ('gpfdist://etl:8081/order_10000000.csv')
FORMAT 'CSV' ( DELIMITER ',' HEADER );
```

Все поля указаны как `TEXT` для минимизации ошибок. В `LOCATION` используется имя контейнера `etl` (в Docker-сети оно разрешается автоматически). `DELIMITER ','` - разделитель в CSV-файле. `HEADER` - первая строка файла содержит заголовки, их нужно пропустить.

5. При помощи запроса извлеките из внешней таблицы несколько строк, и убедитесь в корректности данных. Создайте запрос для извлечения строк внешней таблицы с преобразованием типов в соответствии с данными CSV-файла.

Извлечем из внешней таблицы 10 строк и убедимся что данные корректны:

```
SELECT * FROM external_orders LIMIT 10;
```



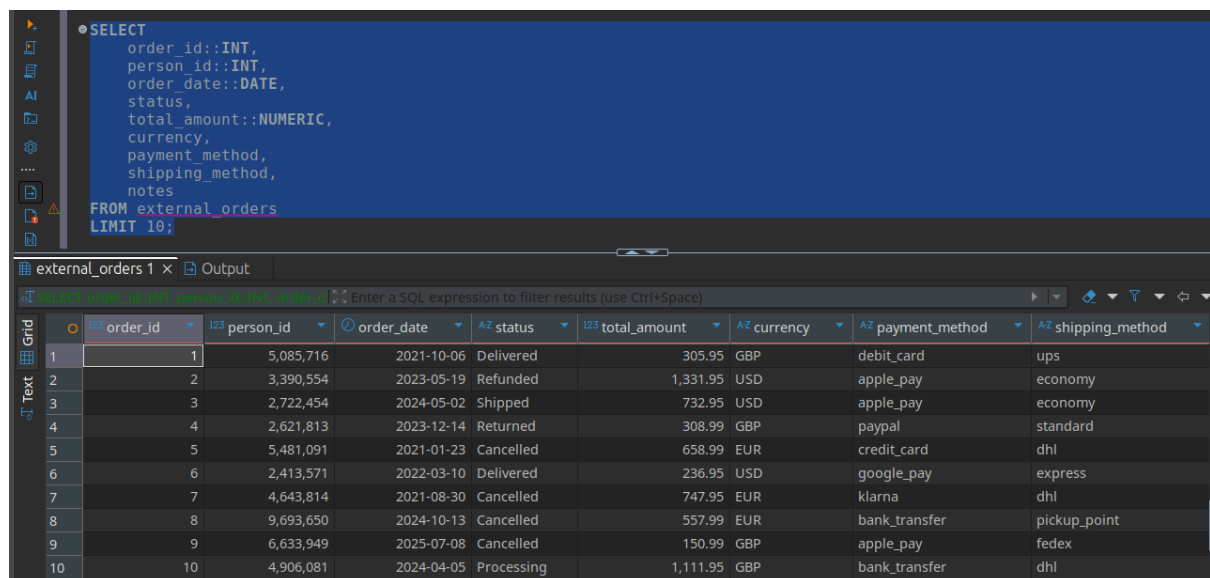
	order_id	person_id	order_date	status	total_amount	currency	payment_method	shipping_method
1	1	5085716	2021-10-06	Delivered	305.95	GBP	debit_card	ups
2	2	3390554	2023-05-19	Refunded	1331.95	USD	apple_pay	economy
3	3	2722454	2024-05-02	Shipped	732.95	USD	apple_pay	economy
4	4	2621813	2023-12-14	Returned	308.99	GBP	paypal	standard
5	5	5481091	2021-01-23	Cancelled	658.99	EUR	credit_card	dhl
6	6	2413571	2022-03-10	Delivered	236.95	USD	google_pay	express
7	7	4643814	2021-08-30	Cancelled	747.95	EUR	klarna	dhl
8	8	9693650	2024-10-13	Cancelled	557.99	EUR	bank_transfer	pickup_point
9	9	6633949	2025-07-08	Cancelled	150.99	GBP	apple_pay	fedex
10	10	4906081	2024-04-05	Processing	1111.95	GBP	bank_transfer	dhl

Данные корректны!

А запрос для извлечения данных из внешней таблицы с преобразованием данных выглядит следующим образом:

```
SELECT
  order_id::INT,
  person_id::INT,
  order_date::DATE,
  status,
  total_amount::NUMERIC,
  currency,
  payment_method,
  shipping_method,
  notes
FROM external_orders
LIMIT 10;
```

Результат следующий:



	order_id	person_id	order_date	status	total_amount	currency	payment_method	shipping_method
1	1	5,085,716	2021-10-06	Delivered	305.95	GBP	debit_card	ups
2	2	3,390,554	2023-05-19	Refunded	1,331.95	USD	apple_pay	economy
3	3	2,722,454	2024-05-02	Shipped	732.95	USD	apple_pay	economy
4	4	2,621,813	2023-12-14	Returned	308.99	GBP	paypal	standard
5	5	5,481,091	2021-01-23	Cancelled	658.99	EUR	credit_card	dhl
6	6	2,413,571	2022-03-10	Delivered	236.95	USD	google_pay	express
7	7	4,643,814	2021-08-30	Cancelled	747.95	EUR	klarna	dhl
8	8	9,693,650	2024-10-13	Cancelled	557.99	EUR	bank_transfer	pickup_point
9	9	6,633,949	2025-07-08	Cancelled	150.99	GBP	apple_pay	fedex
10	10	4,906,081	2024-04-05	Processing	1,111.95	GBP	bank_transfer	dhl

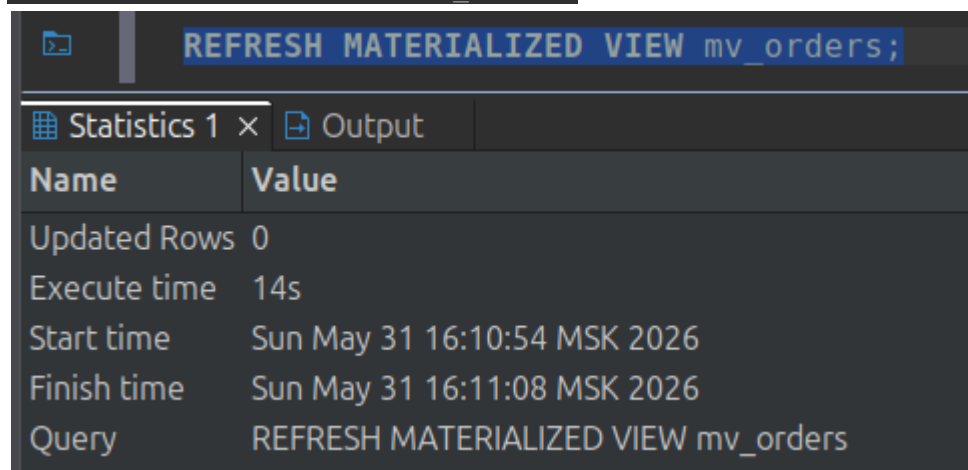
6. Создайте материализованное представление (с опцией with no data), соответствующее структуре CSV-файла, с полями требуемых типов; используйте случайное распределение и колончатый способ хранения. Выполните обновление мат.представления и заметьте затраченное время.

Создадим материализованное представление:

```
CREATE MATERIALIZED VIEW mv_orders
WITH (appendonly = true, orientation = column)
AS
SELECT
    order_id::INT AS order_id,
    person_id::INT AS person_id,
    order_date::DATE AS order_date,
    status,
    total_amount::NUMERIC AS total_amount,
    currency,
    payment_method,
    shipping_method,
    notes
FROM external_orders
WITH NO DATA
DISTRIBUTED RANDOMLY;
```

Далее обновим материализованное представление используя следующий запрос:

```
REFRESH MATERIALIZED VIEW mv_orders;
```



The screenshot shows a database interface with a query editor at the top containing the command 'REFRESH MATERIALIZED VIEW mv_orders;'. Below the editor, there are tabs for 'Statistics 1' and 'Output'. The 'Statistics 1' tab is active, displaying a table with execution metrics.

Name	Value
Updated Rows	0
Execute time	14s
Start time	Sun May 31 16:10:54 MSK 2026
Finish time	Sun May 31 16:11:08 MSK 2026
Query	REFRESH MATERIALIZED VIEW mv_orders

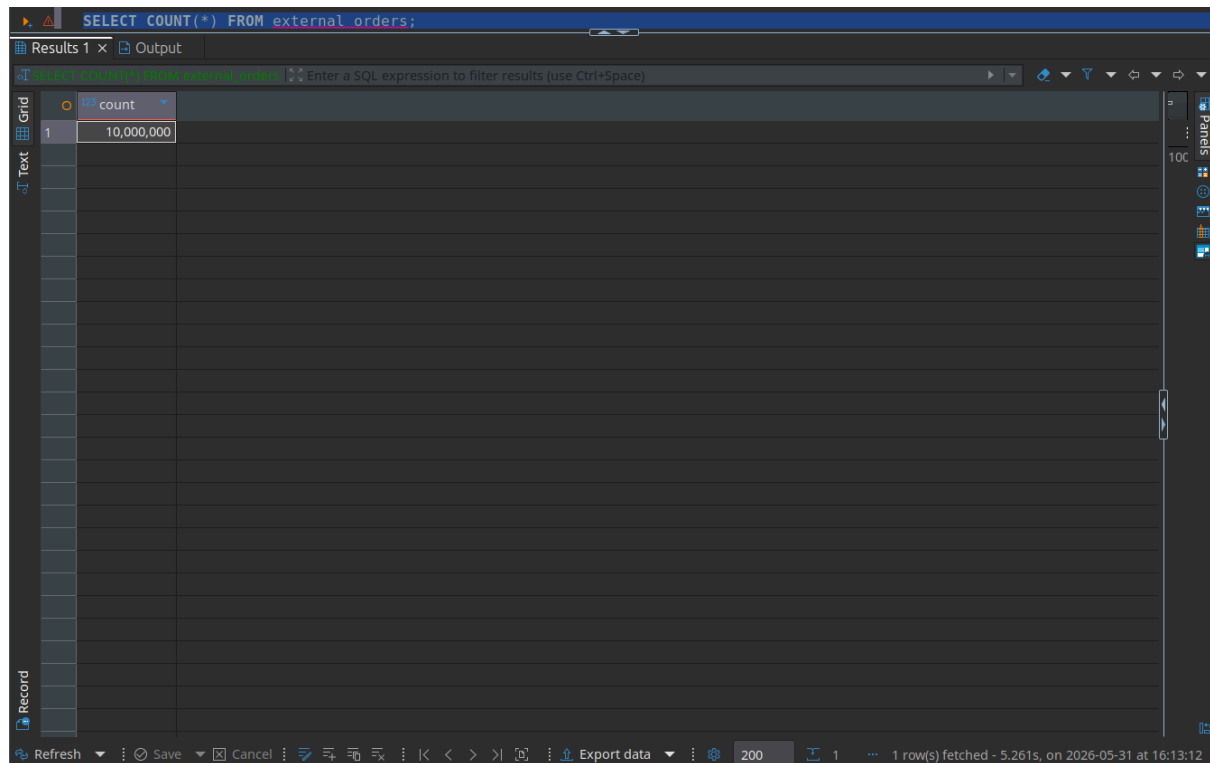
Видно, что время выполнения составило 14 секунд.

7. Посчитайте количество записей во внешней таблице, и в материализованном представлении, и сравните затраченное время.

Сравним время выполнения двух запросов, которые считают количество записей во внешней таблице, и в материализованном представлении:

```
SELECT COUNT(*) FROM external_orders;  
SELECT COUNT(*) FROM mv_orders;
```

Для начала посмотрим сколько будет выполняться запрос к внешней таблице:



The screenshot shows a SQL query execution interface. The query is `SELECT COUNT(*) FROM external_orders;`. The results are displayed in a grid with one row and one column. The column is labeled 'count' and the value is '10,000,000'. The interface also shows a status bar at the bottom indicating '1 row(s) fetched - 5.261s, on 2026-05-31 at 16:13:12'.

count
10,000,000

Справа внизу мы видим, что запрос выполнялся 5.261 секунд.

Теперь сделаем запрос к материализованному представлению:

The screenshot shows a database query results window. The query is `SELECT COUNT(*) FROM mv_orders;`. The results are displayed in a grid with one column labeled 'count' and one row showing the value '10,000,000'. The status bar at the bottom indicates '1 row(s) fetched - 1.021s, on 2026-05-31 at 16:16:58'.

count
10,000,000

Справа внизу видим что запрос выполнялся 1.021 секунд. Это в 5 раз быстрее.

Очевидно, что запрос к материализованному представлению будет значительно быстрее ведь данные уже сохранены внутри Greenplum в колоночном формате. А запрос к внешней таблице будет медленнее так как данные читаются через gpfdist и парсятся.

8. Определите размер данных в мат.представлении.

Определим размер данных в мат.представлении при помощи следующего запроса:

```
SELECT pg_size_pretty(pg_total_relation_size('mv_orders')) AS total_size;
```

Видим что его размер составляет 502 МБ:

The screenshot shows a database query results window. The query is `SELECT pg_size_pretty(pg_total_relation_size('mv_orders')) AS total_size;`. The results are displayed in a grid with one column labeled 'total_size' and one row showing the value '502 MB'. The status bar at the bottom indicates '1 row(s) fetched - 1.021s, on 2026-05-31 at 16:16:58'.

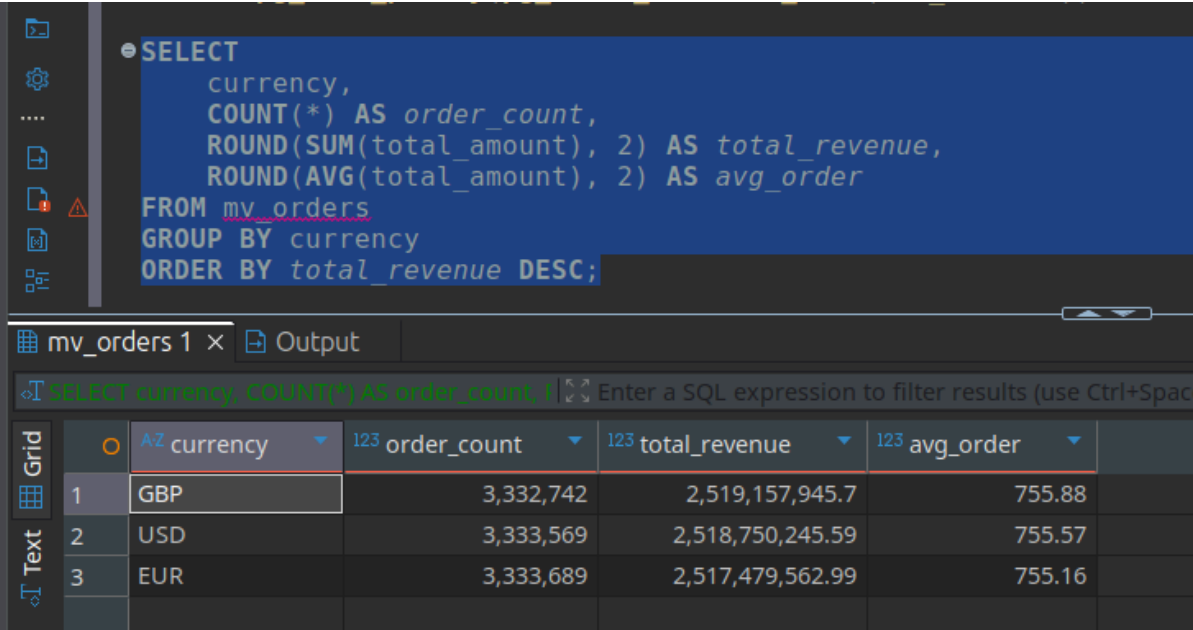
total_size
502 MB

9. Самостоятельно постройте какой-либо запрос к данным из представления.

Построим запрос “Сумма заказов по валютам”:

```
SELECT
  currency,
  COUNT(*) AS order_count,
  ROUND(SUM(total_amount), 2) AS total_revenue,
  ROUND(AVG(total_amount), 2) AS avg_order
FROM mv_orders
GROUP BY currency
ORDER BY total_revenue DESC;
```

Вывод следующий:



The screenshot shows a SQL IDE interface. The top pane displays the SQL query from the previous block. The bottom pane shows the results in a grid view. The grid has 5 columns: currency, order_count, total_revenue, and avg_order. There are 3 rows of data corresponding to GBP, USD, and EUR.

	currency	order_count	total_revenue	avg_order
1	GBP	3,332,742	2,519,157,945.7	755.88
2	USD	3,333,569	2,518,750,245.59	755.57
3	EUR	3,333,689	2,517,479,562.99	755.16

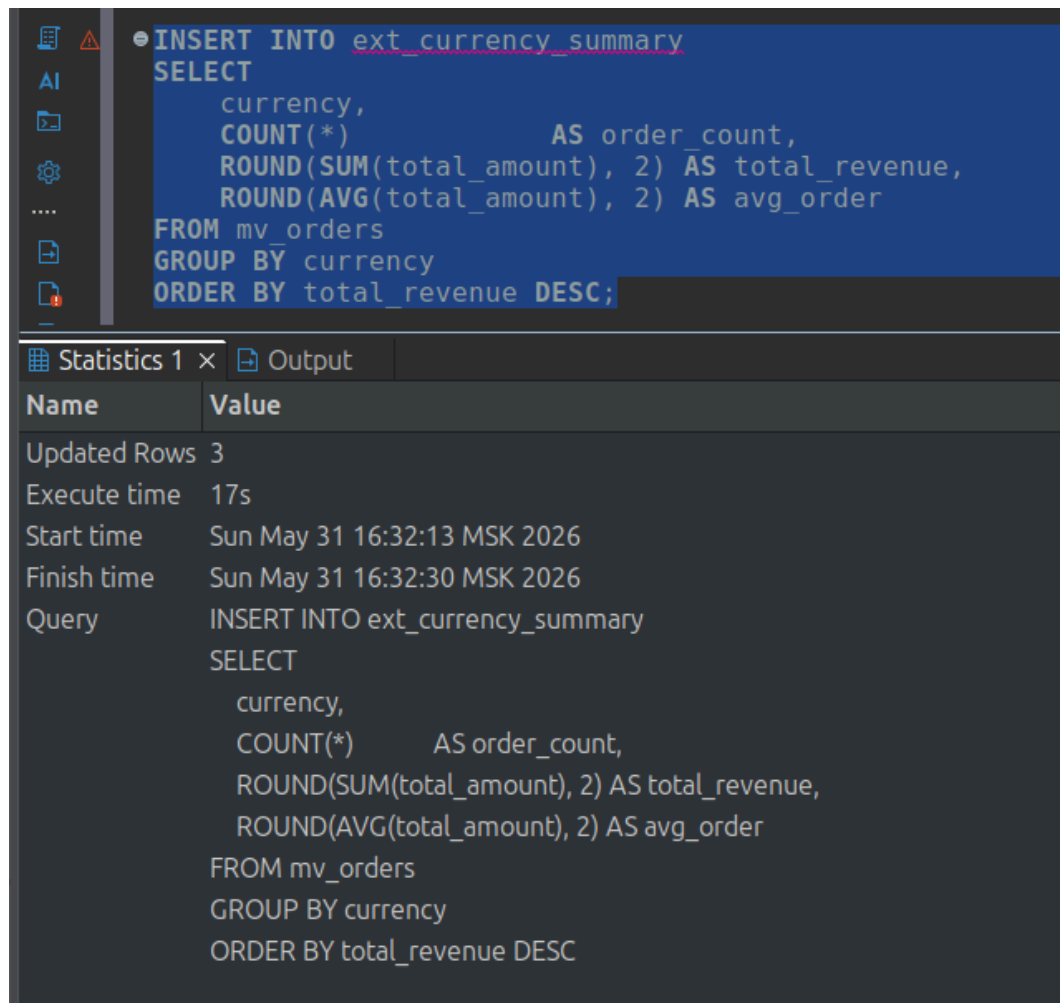
10. На основании разработанного в п.9 запросе создайте external-таблицу для записи данных и загрузите в нее данные. Убедитесь в наличии данных на стороне ETL-сервера.

Запрос для создания EXTERNAL-таблицы для записи выглядит следующим образом:

```
CREATE WRITABLE EXTERNAL TABLE ext_currency_summary (
  currency TEXT,
  order_count BIGINT,
  total_revenue NUMERIC,
  avg_order NUMERIC
)
LOCATION ('gpfdist://etl:8081/currency_summary.csv')
FORMAT 'CSV' (DELIMITER ',');
```


Запрос для загрузки в нее данных выглядит так:

```
INSERT INTO ext_currency_summary
SELECT
    currency,
    COUNT(*) AS order_count,
    ROUND(SUM(total_amount), 2) AS total_revenue,
    ROUND(AVG(total_amount), 2) AS avg_order
FROM mv_orders
GROUP BY currency
ORDER BY total_revenue DESC;
```



The screenshot shows a database IDE interface. The top pane displays a SQL query for inserting data into the `ext_currency_summary` table. The bottom pane shows the execution statistics for this query.

SQL Query:

```
INSERT INTO ext_currency_summary
SELECT
    currency,
    COUNT(*) AS order_count,
    ROUND(SUM(total_amount), 2) AS total_revenue,
    ROUND(AVG(total_amount), 2) AS avg_order
FROM mv_orders
GROUP BY currency
ORDER BY total_revenue DESC;
```

Execution Statistics:

Name	Value
Updated Rows	3
Execute time	17s
Start time	Sun May 31 16:32:13 MSK 2026
Finish time	Sun May 31 16:32:30 MSK 2026
Query	INSERT INTO ext_currency_summary SELECT currency, COUNT(*) AS order_count, ROUND(SUM(total_amount), 2) AS total_revenue, ROUND(AVG(total_amount), 2) AS avg_order FROM mv_orders GROUP BY currency ORDER BY total_revenue DESC

Далее в терминале проверим наличие данных на стороне ETL-сервера:

```
docker exec etl ls -la /data/csv/currency_summary.csv
```

```
> docker exec etl ls -la /data/csv/currency_summary.csv
-rw----- 1 root root 99 May 31 13:32 /data/csv/currency_summary.csv

> ~/gp_cluster
```

Как мы видим данные присутствуют!

11. Создайте какой-либо запрос с фильтрацией к внешней таблицы и изучите план исполнения.

Создал следующий запрос:

```
EXPLAIN ANALYZE
SELECT * FROM external_orders
WHERE status = 'Delivered' AND currency = 'USD';
```

Изучим его план исполнения:

EXPLAIN ANALYZE	
SELECT * FROM external_orders	
WHERE status = 'Delivered' AND currency = 'USD';	
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..602.95 rows=284445 width=72) (actual time=5.168..5812.413 rows=476396 loops=1)
2	-> External Scan on external_orders (cost=0.00..511.00 rows=142223 width=72) (actual time=1.214..5756.717 rows=238751 loops=1)
3	Filter: ((status = 'Delivered'::text) AND (currency = 'USD'::text))
4	Planning time: 5.714 ms
5	(slice0) Executor memory: 162K bytes.
6	(slice1) Executor memory: 207K bytes avg x 2 workers, 207K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 5828.314 ms

Сканирование внешней таблицы и фильтрация выполняются параллельно на 2 сегментах. А мастер получает уже отфильтрованные данные с двух сегментов. Время выполнения 5.8 секунд для 10.000.000 записей.

12. Загрузите использованный CSV-файл каким-либо другим способом, унаследованным от Postgresql (напр. командой copy) и сравнить время загрузки с временем п.6. Команду copy нужно запускать на мастер-сегменте.

Для выполнения этого пункта была создана новая таблица, куда будут загружены данные через COPY. Эта таблица также имеет случайное распределение и является APPEND OPTIMIZED с колоночным хранением:

```
CREATE TABLE orders_copy
(
    order_id INT,
    person_id INT,
    order_date DATE,
    status TEXT,
    total_amount NUMERIC,
    currency TEXT,
    payment_method TEXT,
    shipping_method TEXT,
    notes TEXT
)
WITH (appendonly = true, orientation = column)
DISTRIBUTED RANDOMLY;
```

Далее на сервер с мастер хостом скопируем CSV-файл:

`docker cp order_10000000.csv mdw:/data/orders_10m.csv`

```
> docker cp order_10000000.csv mdw:/data/orders_10m.csv
Successfully copied 796MB (transferred 796MB) to mdw:/data/orders_10m.csv
```

```
> ~Downloads
```

Выполним команду COPY:

```
COPY orders_copy FROM '/data/orders_10m.csv'
CSV DELIMITER ',' HEADER;
```

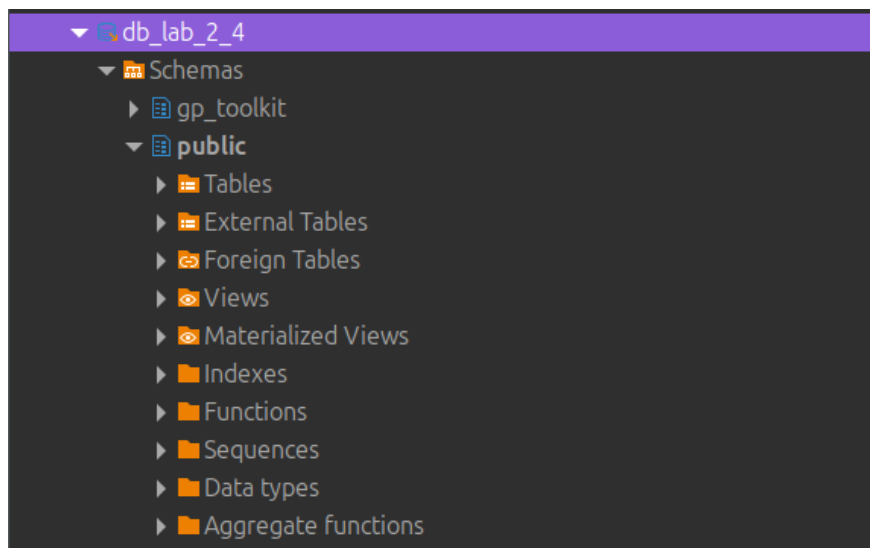
COPY orders_copy FROM '/data/orders_10m.csv' CSV DELIMITER ',' HEADER;	
Statistics 1	Output
Name	Value
Updated Rows	10000000
Execute time	12s
Start time	Sun May 31 17:27:47 MSK 2026
Finish time	Sun May 31 17:28:00 MSK 2026
Query	COPY orders_copy FROM '/data/orders_10m.csv' CSV DELIMITER ',' HEADER

Как мы видим заполнение таблицы через COPY выполнялся 12 секунд. Загрузка в материализованное представление в пункте 6 составляла 14 секунд. То есть теперь у нас на 2 секунды стало быстрее. Время загрузки примерно одинаковое, хотя предполагалось что наверное загрузка через gpfdist должна была выполняться быстрее.

Но вероятно такой результат мы получили из-за того, что на кластере из двух сегментов параллелизм gpfdist даёт минимальный выигрыш, а сетевые обращения к ETL-серверу добавляют задержку. Так еще при этом данные не просто загружаются, но и проходят преобразования типов.

13. Создайте еще одну базу данных в кластере Greenplum, и реализуйте передачу данных какой-либо таблицы из одной БД в другую БД при помощи external-таблиц и протокола gpfdist.

Была создана еще одна база данных в кластере Greenplum:



В ней была создана следующая таблица:

```
CREATE TABLE orders (  
  order_id INT,  
  person_id INT,  
  order_date DATE,  
  status TEXT,  
  total_amount NUMERIC,  
  currency TEXT,  
  payment_method TEXT,  
  shipping_method TEXT,
```

```
    notes TEXT
)
WITH (appendonly = true, orientation = column)
DISTRIBUTED RANDOMLY;
```

Создадим WRITABLE EXTERNAL TABLE в исходной базе данных:

```
CREATE WRITABLE EXTERNAL TABLE external_orders_export (
    order_id INT,
    person_id INT,
    order_date DATE,
    status TEXT,
    total_amount NUMERIC,
    currency TEXT,
    payment_method TEXT,
    shipping_method TEXT,
    notes TEXT
)
LOCATION ('gpfdist://etl:8081/orders_export.csv')
FORMAT 'CSV' (DELIMITER ',');
```

Заполним ее данными:

```
INSERT INTO external_orders_export
SELECT * FROM mv_orders;
```

Создадим READABLE EXTERNAL TABLE в созданной БД:

```
CREATE READABLE EXTERNAL TABLE external_orders_import (
    order_id INT,
    person_id INT,
    order_date DATE,
    status TEXT,
    total_amount NUMERIC,
    currency TEXT,
    payment_method TEXT,
    shipping_method TEXT,
    notes TEXT
)
LOCATION ('gpfdist://etl:8081/orders_export.csv')
FORMAT 'CSV' (DELIMITER ',');
```

Вставим из нее данные в первоначально созданную таблицу orders:

```
INSERT INTO orders
SELECT * FROM external_orders_import;
```

SQL query execution results:

```
INSERT INTO orders
SELECT * FROM external_orders_import;
```

Statistics 1

Name	Value
Updated Rows	10000000
Execute time	11s
Start time	Sun May 31 18:06:32 MSK 2026
Finish time	Sun May 31 18:06:44 MSK 2026
Query	INSERT INTO orders SELECT * FROM external_orders_import

Проверим данные:

SQL query: `SELECT * FROM orders LIMIT 10;`

orders 1

Enter a SQL expression to filter results (use Ctrl+Space)

	order_id	person_id	order_date	status	total_amount	currency	payment_method	shipping_method
1	1,294	7,303,238	2023-01-30	Returned	1,370.95	USD	google_pay	ups
2	1,302	9,762,610	2022-08-11	Cancelled	838.99	EUR	google_pay	dhl
3	1,319	5,342,410	2022-06-28	Processing	365.95	GBP	credit_card	economy
4	1,324	3,299,731	2024-02-21	Shipped	279.95	GBP	google_pay	dhl
5	1,338	7,016,964	2025-07-31	Refunded	1,222.99	EUR	paypal	standard
6	1,349	7,233,369	2021-05-18	Pending	114.99	GBP	debit_card	dhl
7	1,353	4,808,476	2022-07-21	Returned	1,467.95	EUR	credit_card	pickup_point
8	1,368	3,169,008	2022-03-04	Returned	903.95	USD	credit_card	express
9	1,372	718,761	2025-11-09	Shipped	1,075.99	EUR	debit_card	fedex
10	1,379	3,203,578	2025-12-24	Cancelled	92.99	GBP	credit_card	pickup_point

Данные корректны!